

/goal: autonomia com freios — como o Claude Code sabe quando parar

Pesquisa pessoal sobre o comando /goal, disparado pela dúvida da prática "Cansou de Dizer 'Continua'": como o Claude Code decide que uma tarefa autônoma terminou de verdade, e quais travas evitam loops infinitos e faturas surpresa durante a noite.

Fernando Melo · 14 de setembro de 2026

01 · O PROBLEMA

O medo de deixar a IA sozinha à noite

Na prática, Rodrigo descreve uma rotina exaustiva: usar o Claude Code para refatorar e migrar código, mas precisar digitar "continua" a noite inteira porque o agente faz um pedaço, trava e para. Ele ouviu falar do comando /goal, que deixaria o Claude trabalhando sozinho até o fim — mas isso traz duas dúvidas centrais: como o agente sabe que realmente terminou, e o que impede um loop de rodar sem parar e consumir o orçamento enquanto ele dorme.

02 · COMO O /GOAL SABE QUE TERMINOU

Arquitetura de dupla validação

O Claude Code não decide por bom senso que uma tarefa acabou. Ele usa dois papéis separados que se alternam a cada turno:

O Executor (Claude principal)

Planeja, mexe nos arquivos, roda comandos no terminal e tenta resolver o problema — é quem faz o trabalho de fato.

O Avaliador (subagente Haiku)

A cada turno concluído, um modelo rápido e barato (geralmente o Haiku) lê o histórico e pergunta: "a condição pedida foi atingida de forma verificável?" Se não, devolve o que falta e manda o Executor continuar; se sim, encerra e devolve o controle.

É essa conversa entre Executor e Avaliador — não uma sensação de "terminei" do próprio modelo — que sustenta o funcionamento autônomo do /goal.

Travas nativas para não falir

MECANISMO	O QUE FAZ
Detector de falta de progresso	Se o Claude passar vários turnos seguidos sem usar ferramentas (só "conversando" com o Avaliador, sem alterar arquivos ou rodar comandos), o loop para automaticamente e o controle volta para o usuário.
Limite de check-ins ociosos	Enquanto o trabalho roda, o sistema faz verificações de status, mas limita essas checagens automáticas quando a sessão está ociosa (máximo de 3 check-ins), evitando consumo fantasma de tokens.
Erros críticos	Se o terminal retornar um erro de sistema que exija intervenção humana real, o Claude limpa o objetivo e para imediatamente, para não quebrar nada.

Como escrever um /goal seguro

Nunca dar ordens vagas (como "/goal refatore o projeto"). As travas de segurança precisam estar no próprio comando, seguindo três regras:

- 1 Dar um critério de saída numérico ou temporal — um limite de turnos no fim do comando; se chegar lá sem terminar, ele para de qualquer forma.
- 2 Dar uma linha de chegada baseada em teste ou compilação — a IA precisa de um veredito binário do terminal (passou ou falhou) para ter certeza de que terminou.
- 3 Definir limites estritos de escopo — dizer explicitamente o que o agente não pode tocar (ex.: pastas de configuração ou rotas).

Exemplos citados na pesquisa (comandos completos, sem corte)

<pre>/goal Migre o módulo de autenticação para a nova API até que 'npm run build' passe sem erros, OU pare após 15 turnos.</pre>
<pre>/goal Crie os testes unitários para a pasta /src/utils. O objetivo estará concluído quando 'jest --coverage' apontar 80% e todos os testes passarem.</pre>
<pre>/goal Refatore as queries do arquivo user.service.ts para usar Prisma. Restrição: Não altere nenhum arquivo dentro da pasta /config ou /routes.</pre>

Dicas de ouro

Sempre usar Git: fazer git commit ou git stash de tudo antes de disparar o comando. Se a IA fizer besteira durante a noite, um git reset --hard resolve o problema.

De olho nos custos: usar os comandos /usage ou /cost de vez em quando para ver o quanto a sessão já consumiu, e configurar limites de gasto na conta da Anthropic.

O /goal montado durante a prática

Como Rodrigo não tinha uma tarefa real em mente no momento, o exemplo abaixo foi gerado como ilustração de como aplicar as três regras — critério de sucesso testável e limite de turnos — a uma migração de autenticação:

```
/goal Migre todas as funções de autenticação do arquivo auth.js para usar a biblioteca OAuth2.
O objetivo estará concluído quando 'npm run test' passar sem erros. Limite: Máximo de 10
turnos.
```

Autonomia exige contrapartida

O trunfo do /goal é destravar trabalho de fundo com responsabilidade: o desenvolvedor deixa de ser "digitador de continua" e passa a atuar como diretor de engenharia, delegando tarefas longas. Mas isso tem preço — sessões autônomas acumulam contexto e podem consumir muitos tokens em 20-30 turnos.

Bom uso

Critério de sucesso binário e verificável pelo terminal + limite explícito de turnos e de escopo.

Uso arriscado

Objetivo vago, sem comando de validação e sem limite de turnos — o Avaliador (literal) pode manter o loop rodando enquanto tenta adivinhar o que foi pedido.